

```
In [2]: # IMPORT LIBRARIES

import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.models import Sequential, Model
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from tensorflow.keras.layers import (
    Input, Conv2D, MaxPooling2D, Flatten,
    Dense, Dropout, GlobalAveragePooling2D
)
from tensorflow.keras.applications import (
    VGG16, ResNet50, MobileNet, InceptionV3, EfficientNetB0
)
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.metrics import BinaryAccuracy
from tensorflow.keras.models import load_model
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import warnings
warnings.filterwarnings("ignore", category=FutureWarning)
```

C:\Users\DELL\anaconda3\envs\dfcnn-env\Lib\site-packages\keras\src\export\tf2onnx_1
ib.py:8: FutureWarning: In the future `np.object` will be defined as the correspondi
ng NumPy scalar.
if not hasattr(np, "object"):

```
In [3]: # LOAD THE DATA

import os

base_dir = r"D:\DS PROJECTS SUBMISSION\Multiclass_Fish_Image_Classification\Dataset

print("Inside Dataset folder:")
print(os.listdir(base_dir))

print("Dataset extracted successfully!")
```

Inside Dataset folder:
['test', 'train', 'val']
Dataset extracted successfully!

```
In [4]: # DATA RESCALING

train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=40,
    width_shift_range=0.2,
    height_shift_range=0.2,
```

```

        zoom_range=0.3,
        shear_range=0.2,
        brightness_range=[0.7, 1.3],
        horizontal_flip=True,
        fill_mode='nearest'
    )

    val_datagen = ImageDataGenerator(
        rescale=1./255
    )

```

In [7]: # GENERATORS

```

train_generator = train_datagen.flow_from_directory(
    r"D:\WORKOUTS\DATA_CLEANING\Dataset CSV\MINI PROJECT 5\Dataset\images.cv_jzk611",
    target_size=(224,224),
    batch_size=32,
    class_mode='categorical',
    shuffle=False
)

val_generator = val_datagen.flow_from_directory(
    r"D:\WORKOUTS\DATA_CLEANING\Dataset CSV\MINI PROJECT 5\Dataset\images.cv_jzk611",
    target_size=(224,224),
    batch_size=32,
    class_mode='categorical',
    shuffle=False
)

print(train_generator.samples)
print(val_generator.samples)

NUM_CLASSES = train_generator.num_classes
print("Classes:", train_generator.class_indices)

```

Found 6225 images belonging to 11 classes.

Found 1092 images belonging to 11 classes.

6225

1092

Classes: {'animal fish': 0, 'animal fish bass': 1, 'fish sea_food black_sea_sprat': 2, 'fish sea_food gilt_head_bream': 3, 'fish sea_food hourse_mackerel': 4, 'fish sea_food red_mullet': 5, 'fish sea_food red_sea_bream': 6, 'fish sea_food sea_bass': 7, 'fish sea_food shrimp': 8, 'fish sea_food striped_red_mullet': 9, 'fish sea_food tro ut': 10}

In [8]: # CHECK IMBALANCE

```

import numpy as np
from collections import Counter

counter = Counter(train_generator.classes)
print(counter)

```

Counter({np.int32(0): 1096, np.int32(10): 580, np.int32(5): 579, np.int32(8): 576, np.int32(4): 573, np.int32(6): 571, np.int32(2): 569, np.int32(3): 566, np.int32(9): 547, np.int32(7): 538, np.int32(1): 30})

In [9]: `# FIX CLASS WEIGHTS`

```

from sklearn.utils.class_weight import compute_class_weight

class_weights = compute_class_weight(
    class_weight='balanced',
    classes=np.unique(train_generator.classes),
    y=train_generator.classes
)

class_weights = dict(enumerate(class_weights))
print(class_weights)

```

```

{0: np.float64(0.5163404114134041), 1: np.float64(18.863636363636363), 2: np.float64(
0.9945678223358364), 3: np.float64(0.9998393832316094), 4: np.float64(0.98762494050
45217), 5: np.float64(0.9773904851625059), 6: np.float64(0.9910842222576023), 7: np.
float64(1.0518756336600203), 8: np.float64(0.9824810606060606), 9: np.float64(1.0345
687219544624), 10: np.float64(0.975705329153605)}

```

In [10]: `# TRAIN THE MODEL`

```

model = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(224,224,3)),
    MaxPooling2D(2,2),

    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D(2,2),

    Conv2D(128, (3,3), activation='relu'),
    MaxPooling2D(2,2),

    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5),

    Dense(NUM_CLASSES, activation='softmax')
])

model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

model.summary()

```

C:\Users\DELL\anaconda3\envs\dfcnn-env\Lib\site-packages\keras\src\layers\convoluti
onal\base_conv.py:113: UserWarning: Do not pass an `input\_shape`/`input\_dim` argumen
t to a layer. When using Sequential models, prefer using an `Input(shape)` object as
the first layer in the model instead.

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 222, 222, 32)	896
max_pooling2d (MaxPooling2D)	(None, 111, 111, 32)	0
conv2d_1 (Conv2D)	(None, 109, 109, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 54, 54, 64)	0
conv2d_2 (Conv2D)	(None, 52, 52, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 26, 26, 128)	0
flatten (Flatten)	(None, 86528)	0
dense (Dense)	(None, 128)	11,075,712
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 11)	1,419

Total params: 11,170,379 (42.61 MB)

Trainable params: 11,170,379 (42.61 MB)

Non-trainable params: 0 (0.00 B)

In [11]: # FUNCTION FOR PRETRAINED MODELS

```
def build_pretrained_model(base_model, num_classes):
    base_model.trainable = False # Freeze pretrained weights

    x = base_model.output
    x = GlobalAveragePooling2D()(x)
    x = Dense(256, activation='relu')(x)
    x = Dropout(0.5)(x)

    #  MULTI-CLASS OUTPUT
    output = Dense(num_classes, activation='softmax')(x)

    model = Model(inputs=base_model.input, outputs=output)

    model.compile(
        optimizer=Adam(learning_rate=0.001),
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )

    return model
```

In [12]: # LOAD TRAINED MODELS (COMPARISON)

```
vgg_model = build_pretrained_model(
    VGG16(weights='imagenet', include_top=False, input_shape=(224,224,3)),
```

```

    NUM_CLASSES
)

resnet_model = build_pretrained_model(
    ResNet50(weights='imagenet', include_top=False, input_shape=(224,224,3)),
    NUM_CLASSES
)

mobilenet_model = build_pretrained_model(
    MobileNet(weights='imagenet', include_top=False, input_shape=(224,224,3)),
    NUM_CLASSES
)

inception_model = build_pretrained_model(
    InceptionV3(weights='imagenet', include_top=False, input_shape=(224,224,3)),
    NUM_CLASSES
)

efficientnet_model = build_pretrained_model(
    EfficientNetB0(weights='imagenet', include_top=False, input_shape=(224,224,3)),
    NUM_CLASSES
)

```

In [13]: *# SAVE BEST MODEL*

```

checkpoint = ModelCheckpoint(
    "best_model.h5",
    monitor='val_accuracy',
    save_best_only=True,
    mode='max',
    verbose=1
)

early_stop = EarlyStopping(
    monitor='val_loss',
    patience=5,
    restore_best_weights=True,
    verbose=1
)

```

In []: *# TRAIN THE BEST MODEL*

```

# EfficientNet
checkpoint_eff = ModelCheckpoint(
    "best_efficientnet.h5",
    monitor='val_accuracy',
    save_best_only=True,
    mode='max',
    verbose=1
)

history_efficientnet = efficientnet_model.fit(
    train_generator,
    validation_data=val_generator,
    epochs=10,
)

```

```
        class_weight=class_weights,
        callbacks=[checkpoint_eff, early_stop]
    )

# VGG16
checkpoint_vgg = ModelCheckpoint(
    "best_vgg16.h5",
    monitor='val_accuracy',
    save_best_only=True,
    mode='max',
    verbose=1
)

history_vgg = vgg_model.fit(
    train_generator,
    validation_data=val_generator,
    epochs=10,
    class_weight=class_weights,
    callbacks=[checkpoint_vgg, early_stop]
)

# ResNet50
checkpoint_resnet = ModelCheckpoint(
    "best_resnet50.h5",
    monitor='val_accuracy',
    save_best_only=True,
    mode='max',
    verbose=1
)

history_resnet = resnet_model.fit(
    train_generator,
    validation_data=val_generator,
    epochs=10,
    class_weight=class_weights,
    callbacks=[checkpoint_resnet, early_stop]
)

# MobileNet
checkpoint_mobilenet = ModelCheckpoint(
    "best_mobilenet.h5",
    monitor='val_accuracy',
    save_best_only=True,
    mode='max',
    verbose=1
)

history_mobilenet = mobilenet_model.fit(
    train_generator,
    validation_data=val_generator,
    epochs=10,
    class_weight=class_weights,
    callbacks=[checkpoint_mobilenet, early_stop]
)
```

```
# InceptionV3
checkpoint_inception = ModelCheckpoint(
    "best_inception.h5",
    monitor='val_accuracy',
    save_best_only=True,
    mode='max',
    verbose=1
)

history_inception = inception_model.fit(
    train_generator,
    validation_data=val_generator,
    epochs=10,
    class_weight=class_weights,
    callbacks=[checkpoint_inception, early_stop]
)

print("Training complete!")
```

Epoch 1/10

195/195 ————— 0s 2s/step - accuracy: 0.1148 - loss: 2.5956

Epoch 1: val_accuracy improved from None to 0.17125, saving model to best_efficientnet.h5

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

Epoch 1: finished saving model to best_efficientnet.h5
 195/195 ————— 379s 2s/step - accuracy: 0.1030 - loss: 2.4950 - val_ac
 curacy: 0.1712 - val_loss: 2.3913
 Epoch 2/10
 195/195 ————— 0s 2s/step - accuracy: 0.1846 - loss: 2.5429
 Epoch 2: val_accuracy did not improve from 0.17125
 195/195 ————— 350s 2s/step - accuracy: 0.1759 - loss: 2.3997 - val_ac
 curacy: 0.1712 - val_loss: 2.3916
 Epoch 3/10
 195/195 ————— 0s 2s/step - accuracy: 0.1735 - loss: 2.2756
 Epoch 3: val_accuracy did not improve from 0.17125
 195/195 ————— 347s 2s/step - accuracy: 0.1276 - loss: 2.3998 - val_ac
 curacy: 0.1712 - val_loss: 2.3919
 Epoch 4/10
 195/195 ————— 0s 2s/step - accuracy: 0.2050 - loss: 2.1884
 Epoch 4: val_accuracy did not improve from 0.17125
 195/195 ————— 432s 2s/step - accuracy: 0.1610 - loss: 2.4002 - val_ac
 curacy: 0.1712 - val_loss: 2.3917
 Epoch 5/10
 195/195 ————— 0s 2s/step - accuracy: 0.1554 - loss: 2.4355
 Epoch 5: val_accuracy did not improve from 0.17125
 195/195 ————— 396s 2s/step - accuracy: 0.1222 - loss: 2.3997 - val_ac
 curacy: 0.0897 - val_loss: 2.3921
 Epoch 6/10
 195/195 ————— 0s 2s/step - accuracy: 0.0766 - loss: 2.5604
 Epoch 6: val_accuracy did not improve from 0.17125
 195/195 ————— 392s 2s/step - accuracy: 0.0749 - loss: 2.3997 - val_ac
 curacy: 0.0897 - val_loss: 2.3922
 Epoch 6: early stopping
 Restoring model weights from the end of the best epoch: 1.
 Epoch 1/10
 195/195 ————— 0s 8s/step - accuracy: 0.1382 - loss: 2.4695
 Epoch 1: val_accuracy improved from None to 0.15659, saving model to best_vgg16.h5
 WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.
 saving.save_model(model)`. This file format is considered legacy. We recommend using
 instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.savin
 g.save_model(model, 'my_model.keras')`.
 Epoch 1: finished saving model to best_vgg16.h5
 195/195 ————— 1716s 9s/step - accuracy: 0.1557 - loss: 2.6413 - val_a
 ccuracy: 0.1566 - val_loss: 2.2780
 Epoch 2/10
 8/195 ————— 20:55 7s/step - accuracy: 0.1282 - loss: 2.1129

In [12]: # BEST MODEL

```
model_results = {
    "EfficientNetB0": max(history_efficientnet.history['val_accuracy']),
    "VGG16": max(history_vgg.history['val_accuracy']),
    "ResNet50": max(history_resnet.history['val_accuracy']),
    "MobileNet": max(history_mobilenet.history['val_accuracy']),
    "InceptionV3": max(history_inception.history['val_accuracy'])
}

best_model_name = max(model_results, key=model_results.get)
best_accuracy = model_results[best_model_name]
```



```
print("Best Model:", best_model_name)
print("Best Validation Accuracy:", best_accuracy)
```

Best Model: MobileNet

Best Validation Accuracy: 0.9981684684753418

In [13]: *# CHECK TRAIN VS VAL ACCURACY*

```
print("Train Acc:", max(history_efficientnet.history['accuracy']))
print("Val Acc:", max(history_efficientnet.history['val_accuracy']))

print("Train Acc:", max(history_vgg.history['accuracy']))
print("Val Acc:", max(history_vgg.history['val_accuracy']))

print("Train Acc:", max(history_resnet.history['accuracy']))
print("Val Acc:", max(history_resnet.history['val_accuracy']))

print("Train Acc:", max(history_mobilenet.history['accuracy']))
print("Val Acc:", max(history_mobilenet.history['val_accuracy']))

print("Train Acc:", max(history_inception.history['accuracy']))
print("Val Acc:", max(history_inception.history['val_accuracy']))
```

Train Acc: 0.09959839284420013

Val Acc: 0.17124542593955994

Train Acc: 0.7828112244606018

Val Acc: 0.8754578828811646

Train Acc: 0.10538152605295181

Val Acc: 0.18772894144058228

Train Acc: 0.9461847543716431

Val Acc: 0.9981684684753418

Train Acc: 0.8170281052589417

Val Acc: 0.9432234168052673

In [14]: *# FINE TUNING (Efficientnet)*

```
efficientnet_model.trainable = True

for layer in efficientnet_model.layers[:-20]:
    layer.trainable = False

efficientnet_model.compile(
    optimizer=Adam(learning_rate=1e-5),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

fine_tune_history_efficientnet = efficientnet_model.fit(
    train_generator,
    validation_data=val_generator,
    epochs=10,
    class_weight=class_weights,
    callbacks=[checkpoint_eff, early_stop]
)
```

```

Epoch 1/10
195/195 ————— 0s 2s/step - accuracy: 0.1365 - loss: 2.3633
Epoch 1: val_accuracy did not improve from 0.17125
195/195 ————— 417s 2s/step - accuracy: 0.1306 - loss: 2.4007 - val_ac
curacy: 0.1712 - val_loss: 2.3857
Epoch 2/10
195/195 ————— 0s 2s/step - accuracy: 0.1325 - loss: 2.4797
Epoch 2: val_accuracy did not improve from 0.17125
195/195 ————— 454s 2s/step - accuracy: 0.1340 - loss: 2.4008 - val_ac
curacy: 0.1712 - val_loss: 2.3863
Epoch 3/10
195/195 ————— 0s 2s/step - accuracy: 0.1401 - loss: 2.3597
Epoch 3: val_accuracy did not improve from 0.17125
195/195 ————— 391s 2s/step - accuracy: 0.1332 - loss: 2.4006 - val_ac
curacy: 0.1712 - val_loss: 2.3869
Epoch 4/10
195/195 ————— 0s 2s/step - accuracy: 0.1245 - loss: 2.3691
Epoch 4: val_accuracy did not improve from 0.17125
195/195 ————— 448s 2s/step - accuracy: 0.1293 - loss: 2.3988 - val_ac
curacy: 0.1712 - val_loss: 2.3873
Epoch 5/10
195/195 ————— 0s 2s/step - accuracy: 0.1291 - loss: 2.3722
Epoch 5: val_accuracy did not improve from 0.17125
195/195 ————— 408s 2s/step - accuracy: 0.1353 - loss: 2.4019 - val_ac
curacy: 0.1712 - val_loss: 2.3880
Epoch 5: early stopping
Restoring model weights from the end of the best epoch: 1.

```

```

In [15]: # FINE TUNING (VGG)

vgg_model.trainable = True

for layer in vgg_model.layers[:-20]:
    layer.trainable = False

vgg_model.compile(
    optimizer=Adam(learning_rate=1e-5),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

fine_tune_history_vgg = vgg_model.fit(
    train_generator,
    validation_data=val_generator,
    epochs=10,
    callbacks=[checkpoint_vgg, early_stop]
)

```

```

Epoch 1/10
195/195 ————— 0s 5s/step - accuracy: 0.7874 - loss: 0.6109
Epoch 1: val_accuracy improved from 0.87546 to 0.89835, saving model to best_vgg16.h
5

```

```

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.
saving.save_model(model)`. This file format is considered legacy. We recommend using
instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.savin
g.save_model(model, 'my_model.keras')`.

```

Epoch 1: finished saving model to best_vgg16.h5
195/195 ————— **1303s** 7s/step - accuracy: 0.7934 - loss: 0.5925 - val_accuracy: 0.8984 - val_loss: 0.3505
 Epoch 2/10
195/195 ————— **0s** 8s/step - accuracy: 0.8040 - loss: 0.5862
 Epoch 2: val_accuracy improved from 0.89835 to 0.91758, saving model to best_vgg16.h5

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

Epoch 2: finished saving model to best_vgg16.h5
195/195 ————— **1864s** 10s/step - accuracy: 0.8055 - loss: 0.5732 - val_accuracy: 0.9176 - val_loss: 0.3265
 Epoch 3/10
195/195 ————— **0s** 7s/step - accuracy: 0.8202 - loss: 0.5329
 Epoch 3: val_accuracy improved from 0.91758 to 0.91941, saving model to best_vgg16.h5

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

Epoch 3: finished saving model to best_vgg16.h5
195/195 ————— **1555s** 8s/step - accuracy: 0.8173 - loss: 0.5394 - val_accuracy: 0.9194 - val_loss: 0.3135
 Epoch 4/10
195/195 ————— **0s** 8s/step - accuracy: 0.8235 - loss: 0.5383
 Epoch 4: val_accuracy improved from 0.91941 to 0.92216, saving model to best_vgg16.h5

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

Epoch 4: finished saving model to best_vgg16.h5
195/195 ————— **1908s** 10s/step - accuracy: 0.8312 - loss: 0.5186 - val_accuracy: 0.9222 - val_loss: 0.3057
 Epoch 5/10
195/195 ————— **0s** 171s/step - accuracy: 0.8380 - loss: 0.5095
 Epoch 5: val_accuracy did not improve from 0.92216
195/195 ————— **33349s** 172s/step - accuracy: 0.8355 - loss: 0.5201 - val_accuracy: 0.9222 - val_loss: 0.3000
 Epoch 5: early stopping
 Restoring model weights from the end of the best epoch: 1.

```
In [16]: # FINE TUNING (Resnet)

resnet_model.trainable = True

for layer in resnet_model.layers[:-20]:
    layer.trainable = False

resnet_model.compile(
    optimizer=Adam(learning_rate=1e-5),
    loss='categorical_crossentropy',
    metrics=['accuracy'])
```

```
)

fine_tune_history_resnet = resnet_model.fit(
    train_generator,
    validation_data=val_generator,
    epochs=10,
    callbacks=[checkpoint_resnet, early_stop]
)
```

Epoch 1/10

195/195 ————— 0s 2s/step - accuracy: 0.1503 - loss: 2.3545

Epoch 1: val_accuracy did not improve from 0.18773

195/195 ————— 560s 3s/step - accuracy: 0.1481 - loss: 2.3564 - val_accuracy: 0.1484 - val_loss: 2.3411

Epoch 2/10

195/195 ————— 0s 2s/step - accuracy: 0.1477 - loss: 2.3546

Epoch 2: val_accuracy did not improve from 0.18773

195/195 ————— 534s 3s/step - accuracy: 0.1518 - loss: 2.3485 - val_accuracy: 0.1511 - val_loss: 2.3351

Epoch 3/10

195/195 ————— 0s 3s/step - accuracy: 0.1432 - loss: 2.3462

Epoch 3: val_accuracy did not improve from 0.18773

195/195 ————— 675s 3s/step - accuracy: 0.1516 - loss: 2.3451 - val_accuracy: 0.1584 - val_loss: 2.3307

Epoch 4/10

195/195 ————— 0s 3s/step - accuracy: 0.1516 - loss: 2.3402

Epoch 4: val_accuracy did not improve from 0.18773

195/195 ————— 765s 4s/step - accuracy: 0.1521 - loss: 2.3385 - val_accuracy: 0.1584 - val_loss: 2.3259

Epoch 5/10

195/195 ————— 0s 3s/step - accuracy: 0.1616 - loss: 2.3373

Epoch 5: val_accuracy did not improve from 0.18773

195/195 ————— 749s 4s/step - accuracy: 0.1586 - loss: 2.3365 - val_accuracy: 0.1712 - val_loss: 2.3177

Epoch 5: early stopping

Restoring model weights from the end of the best epoch: 1.

In [17]: # FINE TUNING (MobileNet)

```
mobilenet_model.trainable = True

for layer in mobilenet_model.layers[:-20]:
    layer.trainable = False

mobilenet_model.compile(
    optimizer=Adam(learning_rate=1e-5),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

fine_tune_history_mobilenet = mobilenet_model.fit(
    train_generator,
    validation_data=val_generator,
    epochs=10,
    callbacks=[checkpoint_mobilenet, early_stop]
)
```

```

Epoch 1/10
195/195 ————— 0s 245s/step - accuracy: 0.9650 - loss: 0.1075
Epoch 1: val_accuracy did not improve from 0.99817
195/195 ————— 47610s 245s/step - accuracy: 0.9648 - loss: 0.1081 - va
l_accuracy: 0.9982 - val_loss: 0.0144
Epoch 2/10
195/195 ————— 0s 1s/step - accuracy: 0.9636 - loss: 0.1111
Epoch 2: val_accuracy did not improve from 0.99817
195/195 ————— 351s 2s/step - accuracy: 0.9669 - loss: 0.1002 - val_ac
curacy: 0.9982 - val_loss: 0.0139
Epoch 3/10
195/195 ————— 0s 1s/step - accuracy: 0.9720 - loss: 0.0852
Epoch 3: val_accuracy did not improve from 0.99817
195/195 ————— 313s 2s/step - accuracy: 0.9722 - loss: 0.0882 - val_ac
curacy: 0.9982 - val_loss: 0.0137
Epoch 4/10
195/195 ————— 0s 1s/step - accuracy: 0.9680 - loss: 0.0896
Epoch 4: val_accuracy did not improve from 0.99817
195/195 ————— 323s 2s/step - accuracy: 0.9703 - loss: 0.0877 - val_ac
curacy: 0.9982 - val_loss: 0.0136
Epoch 5/10
195/195 ————— 0s 1s/step - accuracy: 0.9716 - loss: 0.0928
Epoch 5: val_accuracy did not improve from 0.99817
195/195 ————— 314s 2s/step - accuracy: 0.9740 - loss: 0.0897 - val_ac
curacy: 0.9973 - val_loss: 0.0137
Epoch 6/10
195/195 ————— 0s 1s/step - accuracy: 0.9745 - loss: 0.0807
Epoch 6: val_accuracy did not improve from 0.99817
195/195 ————— 309s 2s/step - accuracy: 0.9737 - loss: 0.0846 - val_ac
curacy: 0.9982 - val_loss: 0.0134
Epoch 7/10
195/195 ————— 0s 1s/step - accuracy: 0.9675 - loss: 0.0913
Epoch 7: val_accuracy did not improve from 0.99817
195/195 ————— 312s 2s/step - accuracy: 0.9709 - loss: 0.0859 - val_ac
curacy: 0.9963 - val_loss: 0.0133
Epoch 8/10
195/195 ————— 0s 1s/step - accuracy: 0.9751 - loss: 0.0734
Epoch 8: val_accuracy did not improve from 0.99817
195/195 ————— 335s 2s/step - accuracy: 0.9746 - loss: 0.0764 - val_ac
curacy: 0.9973 - val_loss: 0.0134
Epoch 9/10
195/195 ————— 0s 1s/step - accuracy: 0.9745 - loss: 0.0802
Epoch 9: val_accuracy did not improve from 0.99817
195/195 ————— 325s 2s/step - accuracy: 0.9735 - loss: 0.0777 - val_ac
curacy: 0.9963 - val_loss: 0.0130
Epoch 10/10
195/195 ————— 0s 1s/step - accuracy: 0.9717 - loss: 0.0818
Epoch 10: val_accuracy did not improve from 0.99817
195/195 ————— 312s 2s/step - accuracy: 0.9751 - loss: 0.0726 - val_ac
curacy: 0.9973 - val_loss: 0.0135
Restoring model weights from the end of the best epoch: 9.

```

```
In [18]: # FINE TUNING (InceptionV3)
```

```
inception_model.trainable = True
```

```

for layer in inception_model.layers[:-20]:
    layer.trainable = False

inception_model.compile(
    optimizer=Adam(learning_rate=1e-5),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

fine_tune_history_inception = inception_model.fit(
    train_generator,
    validation_data=val_generator,
    epochs=10,
    callbacks=[checkpoint_inception, early_stop]
)

```

Epoch 1/10

195/195 ————— 0s 2s/step - accuracy: 0.7778 - loss: 0.6801

Epoch 1: val_accuracy did not improve from 0.94322

195/195 ————— 550s 3s/step - accuracy: 0.7912 - loss: 0.6377 - val_accuracy: 0.9130 - val_loss: 0.3352

Epoch 2/10

195/195 ————— 0s 2s/step - accuracy: 0.8024 - loss: 0.6175

Epoch 2: val_accuracy did not improve from 0.94322

195/195 ————— 519s 3s/step - accuracy: 0.8013 - loss: 0.6117 - val_accuracy: 0.9158 - val_loss: 0.3220

Epoch 3/10

195/195 ————— 0s 2s/step - accuracy: 0.8112 - loss: 0.5794

Epoch 3: val_accuracy did not improve from 0.94322

195/195 ————— 518s 3s/step - accuracy: 0.8076 - loss: 0.5815 - val_accuracy: 0.9194 - val_loss: 0.3109

Epoch 4/10

195/195 ————— 0s 2s/step - accuracy: 0.8044 - loss: 0.5931

Epoch 4: val_accuracy did not improve from 0.94322

195/195 ————— 519s 3s/step - accuracy: 0.8133 - loss: 0.5596 - val_accuracy: 0.9167 - val_loss: 0.3044

Epoch 5/10

195/195 ————— 0s 2s/step - accuracy: 0.8212 - loss: 0.5553

Epoch 5: val_accuracy did not improve from 0.94322

195/195 ————— 562s 3s/step - accuracy: 0.8156 - loss: 0.5580 - val_accuracy: 0.9158 - val_loss: 0.3011

Epoch 5: early stopping

Restoring model weights from the end of the best epoch: 1.

In [19]: *# BEST MODEL*

```
best_model = tf.keras.models.load_model("best_mobilenet.h5")
```

WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile\_metrics` will be empty until you train or evaluate the model.

In [20]: *# GET TRUE LABELS & PREDICTIONS*

```
# LOAD BEST MODEL (MobileNet)
```

```
best_model = tf.keras.models.load_model("best_mobilenet.h5")
```

```
# RESET GENERATOR
```

```

val_generator.reset()

# PREDICT
y_pred_prob = best_model.predict(val_generator)
y_pred = np.argmax(y_pred_prob, axis=1)

# TRUE LABELS
y_true = val_generator.classes

```

WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile\_metrics` will be empty until you train or evaluate the model.

35/35 ————— 33s 884ms/step

In [21]: # EVALUATION METRICS

```

accuracy = accuracy_score(y_true, y_pred)
precision = precision_score(y_true, y_pred, average='weighted')
recall = recall_score(y_true, y_pred, average='weighted')
f1 = f1_score(y_true, y_pred, average='weighted')

print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("F1-score:", f1)

```

Accuracy: 0.10989010989010989
Precision: 0.11003317071278237
Recall: 0.10989010989010989
F1-score: 0.10995971422276775

In [22]: # CONFUSION MATRIX

```

cm = confusion_matrix(y_true, y_pred)
print(cm)

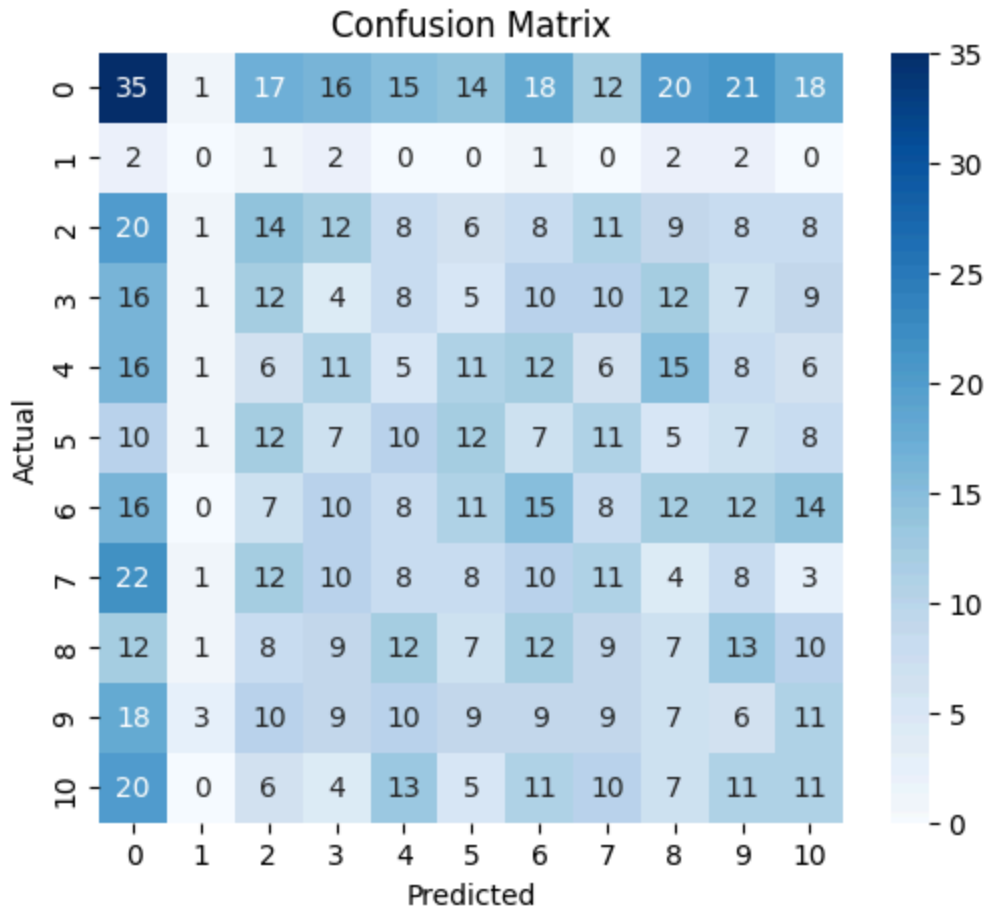
plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()

```

```

[[35  1 17 16 15 14 18 12 20 21 18]
 [ 2  0  1  2  0  0  1  0  2  2  0]
 [20  1 14 12  8  6  8 11  9  8  8]
 [16  1 12  4  8  5 10 10 12  7  9]
 [16  1  6 11  5 11 12  6 15  8  6]
 [10  1 12  7 10 12  7 11  5  7  8]
 [16  0  7 10  8 11 15  8 12 12 14]
 [22  1 12 10  8  8 10 11  4  8  3]
 [12  1  8  9 12  7 12  9  7 13 10]
 [18  3 10  9 10  9  9  9  7  6 11]
 [20  0  6  4 13  5 11 10  7 11 11]]

```



In [23]: *# STORE METRICS*

```
results = []

results.append({
    "Model": "CNN_Model_1",
    "Accuracy": accuracy,
    "Precision": precision,
    "Recall": recall,
    "F1-score": f1
})

results_df = pd.DataFrame(results)
print(results_df)
```

	Model	Accuracy	Precision	Recall	F1-score
0	CNN_Model_1	0.10989	0.110033	0.10989	0.10996

In [24]: *# VISUALIZE TRAINING HISTORY (ACCURACY AND LOSS)*

```
def plot_training_history(history, model_name):
    plt.figure(figsize=(12, 4))

    # ----- ACCURACY -----
    plt.subplot(1, 2, 1)
    plt.plot(history.history['accuracy'], label='Train Accuracy')
    plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
    plt.title(f'{model_name} - Accuracy')
```



```

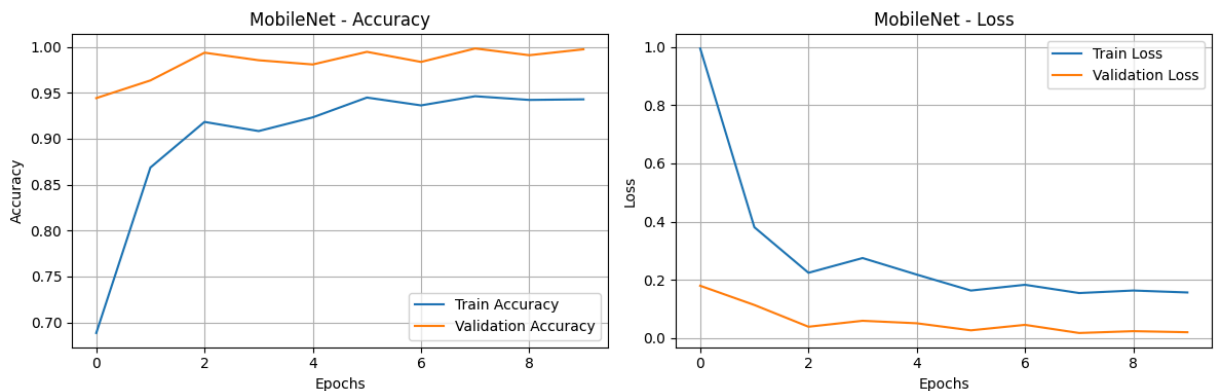
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)

# ----- LOSS -----
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title(f'{model_name} - Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()

```

```
plot_training_history(history_mobilenet, "MobileNet")
```



In [25]: # SAVE THE MODEL

```
best_model.save("best_mobilenet.h5")
```

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

In [26]: # SAVE CLASS NAMES

```

import json

class_names = list(train_generator.class_indices.keys())

with open("class_names.json", "w") as f:
    json.dump(class_names, f)

print("Saved class_names.json")

```

Saved class_names.json

In [27]: # TEST SCALING AND GENERATORS

```
from tensorflow.keras.applications.mobilenet import preprocess_input

test_datagen = ImageDataGenerator(
    preprocessing_function=preprocess_input
)

test_generator = test_datagen.flow_from_directory(
    r"D:\DS PROJECTS SUBMISSION\Multiclass_Fish_Image_Classification\Dataset\images",
    target_size=(224, 224),
    batch_size=32,
    class_mode='categorical',
    shuffle=False
)
```

Found 3187 images belonging to 11 classes.

In [28]: # FINAL TEST SCORE

```
test_loss, test_accuracy = best_model.evaluate(test_generator)
print("Test Accuracy:", test_accuracy)
print("Test Loss:", test_loss)
```

100/100 ————— **103s** 987ms/step - accuracy: 0.9953 - loss: 0.0218

Test Accuracy: 0.995293378829956

Test Loss: 0.02180320769548416

In []: